Reptilian 37 – Documentation

Reptilian 37 is the next-generation educational OS for COP4600 that can run efficiently on a RISC-V SBC. In our case, the device of choice is the Bananapi F3 (BPI-F3). it will:

- Offer a pared-down, secure distribution (Debian) with a reasonably sized footprint.
- Provide exam security features such as kernel signature verification and trusted computing elements.
- Enable dual use: daily assignments in a lean environment and tamper-resistant in-class practical assessments.

If you are a prospective developer of Reptilian 37, please reference the following documentation:

https://blog.bitsofnetworks.org/riscv-upstream-bpi-f3-part1-hardware/

https://blog.bitsofnetworks.org/riscv-upstream-bpi-f3-part2-debian/

https://developer.spacemit.com/documentation?token=GAmMwCrRUiZAo5kXKUQcNNacnwb&type=pdf

https://docs.banana-pi.org/en/BPI-F3/BananaPi_BPI-F3

https://docs.u-boot.org/en/stable/board/spacemit/bananapi-f3.html

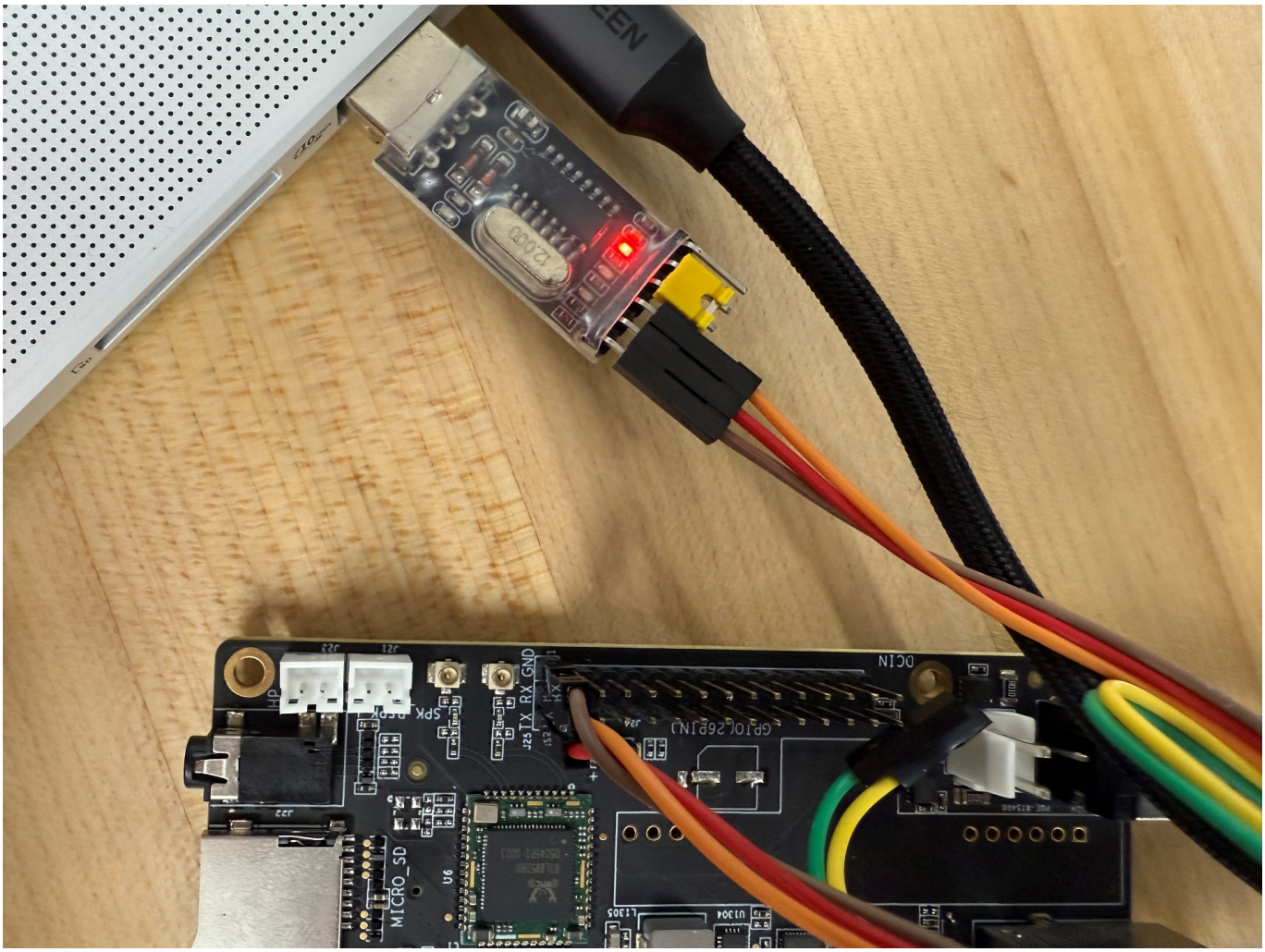
---

USERS

```
root : reptilian
nick17 : reptilian
```

UART

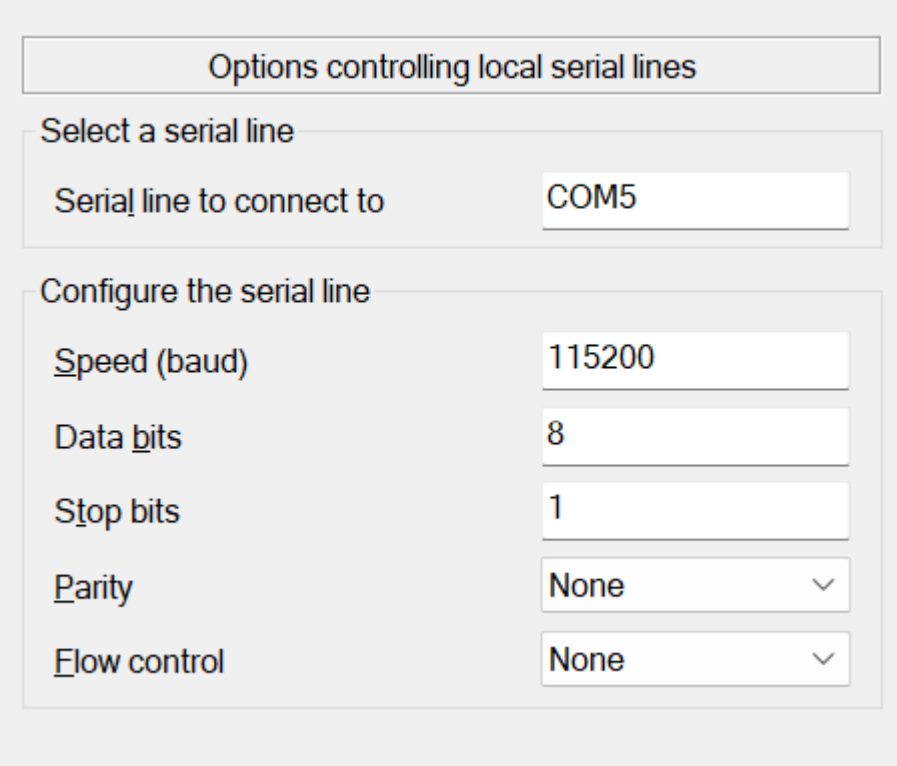
Users can access **Reptilian 37** over UART via a small UART-to-USB breakout board.



UART is invaluable because it:

- Displays the full boot process
- Provides a recovery path if the system is misconfigured
- Gives BusyBox shell access if something goes wrong
- Works as a fallback to SSH if networking breaks

To make use of UART, use the following settings:



Be sure the COM port matches the one used on your host PC. This information can be found in Device Manager.

NETWORKING

Networking on **Reptilian 37** is really convoluted. Here's the breakdown:

end0 (RJ45 1)

- DHCP enabled
- Used for internet access
- SSH access unreliable because the IP changes

end1 (RJ45 2)

- Static IP: `192.168.137.7`
- Chosen for compatibility with Windows Internet Connection Sharing
- Reliable for SSH access

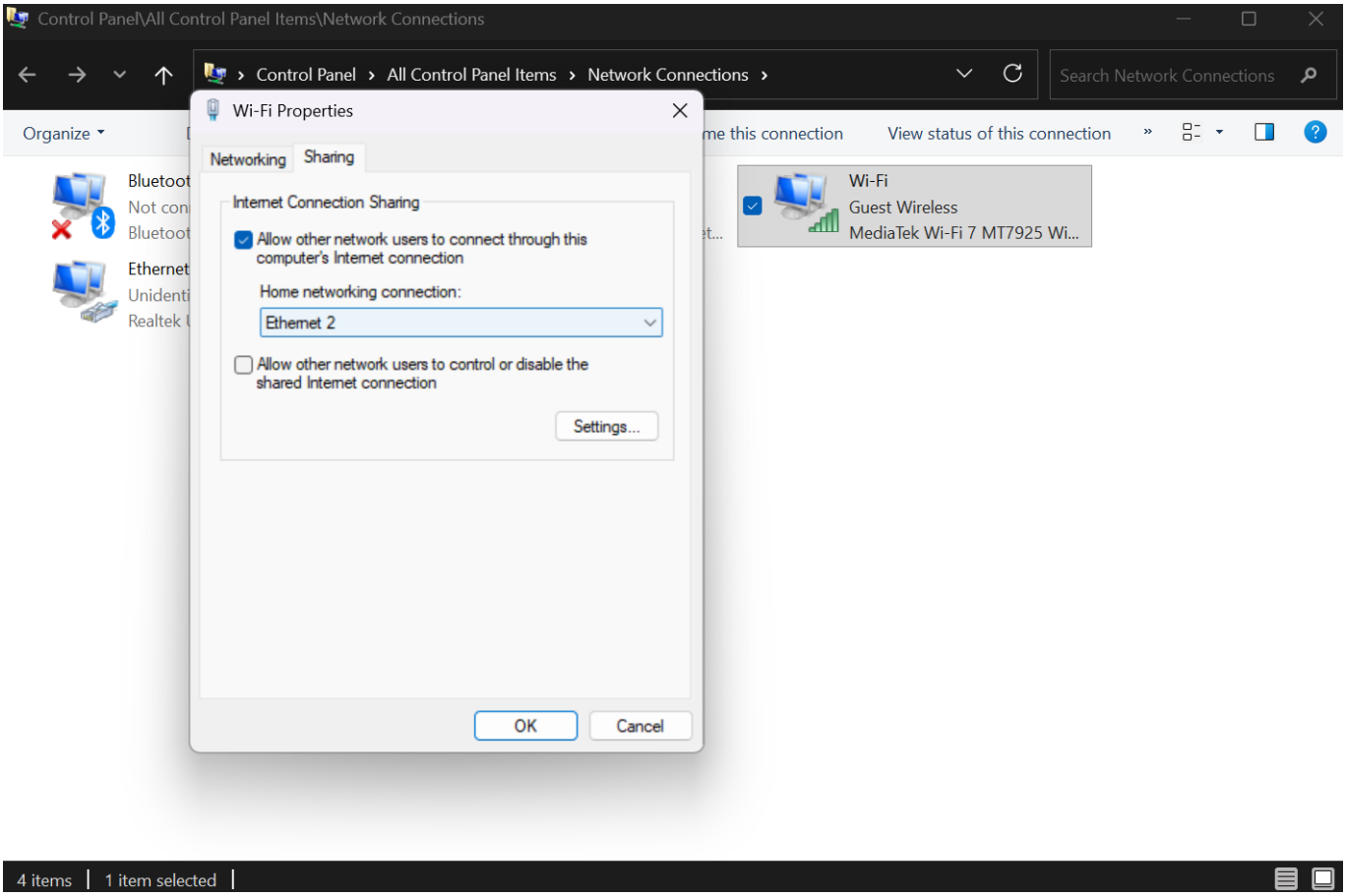
wlan0 (RTL8852BS combo Wifi/BT module)

- DHCP enabled
- Manually set up SSID/password in `/etc/network/interfaces`

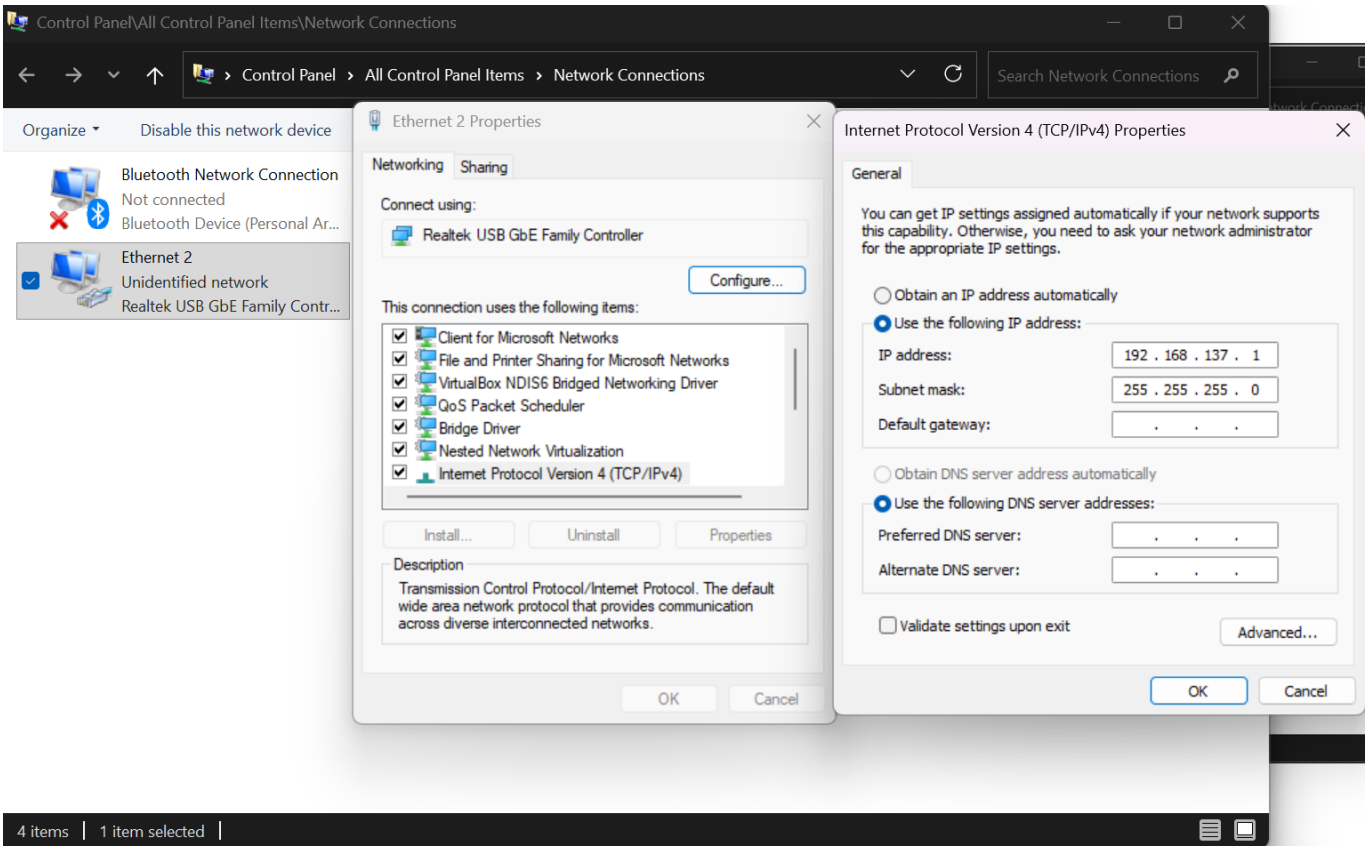
Enabling Windows Internet Sharing

Windows ICS (Internet Connection Sharing) allows you to share your device's internet connection over an ethernet cable, which can be extremely useful if the Wifi card on the Bananapi F3 is unresponsive or misconfigured (as it was for the majority of the Fall 2025 development cycle).

1. Press **Win + R**
2. Type `control ncpa.cpl`
3. Right-click your internet adapter → **Properties**
4. Go to the **Sharing** tab
5. Enable **Allow other network users to connect…**
6. Select the Ethernet adapter connected to the board



You may also need to configure a static IPv4 setup:



/etc/network/interfaces

```
# interfaces(5) file used by ifup(8) and ifdown(8)
# Include files from /etc/network/interfaces.d:
source /etc/network/interfaces.d/*

# end0: DHCP from LAN or router
auto end0
iface end0 inet dhcp
        metric 100

# end1: Static IP for direct connection bananapi <-> PC
auto end1
iface end1 inet static
        address 192.168.137.7
        netmask 255.255.255.0
        gateway 192.168.137.1
        dns-nameservers 8.8.8.8
        metric 200

auto wlan0
iface wlan0 inet dhcp
        #address 192.168.1.37
        #netmask 255.255.255.0
        #gateway 192.168.1.1
        #dns-nameservers 8.8.8.8
```

```
        wpa-ssid "someLizardReference"
        wpa-psk "passssword"
        metric 100
```

/etc/resolv.conf

This file has been modified to use Google DNS:

```
# Generated by dhcpcd
# /etc/resolv.conf.head can replace this line
# /etc/resolv.conf.tail can replace this line
nameserver 8.8.8.8
```

WiFi Notes

The BananaPi F3 uses an **RTL8852BS** combination wireless/bluetooth module.

It took me way too long to figure out that I just needed to probe the module to get Wifi working:

```
modprobe 8852
```

Thankfully, users don't have to worry about this. The module is now probed on boot automatically:

```
echo "8852" > /etc/modules-load.d/rtl8852bs.conf
```

SECURITY

A folder **/root/fitboot** contains resources for FIT kernel signing. The following files need to be copied into the directory in order to create the signed kernel:

- /boot/vmlinuz-6.6.63
- /boot/initrd.img-6.6.63
- /boot/spacemit/6.6.63/k1-x_deb1.dtb

Generate Signing Keys

```
openssl genpkey -algorithm RSA -pkeyopt rsa_keygen_bits:2048 -out
uboot_signing_key.pem
openssl req -new -x509 -key uboot_signing_key.pem -out uboot_signing_key.crt -
subj "/CN=BPI-F3-FIT-Signing/"
```

`fitboot/kernel.its` — FIT Description File

```
/dts-v1/;

/ {
    description = "BPI-F3 FIT Signed Kernel";

    images {

        kernel {
            description = "Linux kernel";
            data = /incbin/("vmlinuz-6.6.63");
            type = "kernel";
            arch = "riscv";
            os = "linux";
            compression = "none";
            load = <0x600000>;
            entry = <0x600000>;
            hash-1 {
                algo = "sha256";
            };
        };

        fdt {
            description = "Flattened Device Tree";
            data = /incbin/("k1-x_deb1.dtb");
            type = "flat_dt";
            arch = "riscv";
            os = "linux";
            compression = "none";
            hash-1 {
                algo = "sha256";
            };
        };

        ramdisk {
            description = "Initramfs";
```

```
            data = /incbin/("initrd.img-6.6.63");
            type = "ramdisk";
            arch = "riscv";
            os = "linux";
            compression = "zstd";
            hash-1 {
                algo = "sha256";
            };
        };
    };

    configurations {
        default = "conf";

        conf {
            description = "BPI-F3 Signed Boot Config";
            kernel = "kernel";
            fdt = "fdt";
            ramdisk = "ramdisk";

            signature {
                algo = "sha256,rsa2048";
                key-name-hint = "uboot_signing_key";
                sign-images = "kernel", "fdt", "ramdisk";
                required = "conf";
            };
        };
    };
};
```

Creating an Unsigned FIT

```
root@reptilian37:~/fitboot# mkimage -f kernel.its kernel_unsigned.itb
```

Signing the FIT

```
root@reptilian37:~/fitboot# mkimage -f kernel.its -k . -K kernel_unsigned.itb -r
kernel_signed.itb
```

Boot Environment – `/boot/env-k1.txt`

The boot environment must be modified to load the kernel from eMMC's boot partition, `/dev/mmcblk2p5`:

```
knl_name=vmlinuz-6.6.63

ramdisk_name=initrd.img-6.6.63

dtb_dir=spacemit/6.6.63

bootargs=earlyprintk quiet splash plymouth.ignore-serial-consoles
plymouth.prefer-fbcon clk_ignore_unused swiotlb=65536
workqueue.default_affinity_scope=system rootwait rootfstype=ext4
root=UUID=a7e88809-e382-4b28-acd4-2ed57abd378f earlycon=sbi
console=ttyS0,115200n8 loglevel=8 rdinit=/init

bootcmd=mmc dev 2; ext4load mmc 2:5 ${kernel_addr_r} kernel_signed.itb; bootm
${kernel_addr_r}
```

Example UART Boot Log with Signature Checking

If everything goes as planned, you should see the hash checks being performed over the UART boot log:

```
sys: 0x0
try sd...
bm:3
ERROR:   CMD8
ERROR:   sd f! l:76
bm:0
j...

U-Boot SPL 2022.10spacemit-dirty (Sep 09 2024 - 03:41:00 +0000)
[   0.166] DDR type LPDDR4X
[   0.176] lpddr4_silicon_init consume 10ms
[   0.177] Change DDR data rate to 2400MT/s
[   0.282] Boot from fit configuration k1-x_deb1
[   0.284] ## Checking hash(es) for config conf_1 ... OK
[   0.289] ## Checking hash(es) for Image uboot ... crc32+ OK
[   0.301] ## Checking hash(es) for Image fdt_1 ... crc32+ OK
[   0.322] ## Checking hash(es) for config config_1 ... OK
[   0.324] ## Checking hash(es) for Image opensbi ... crc32+ OK
[   0.362]
```

```
U-Boot 2022.10spacemit-dirty (Aug 15 2025 - 03:33:48 +0000), Build: jenkins-BSP-
build-deb-751

...

[   1.050] Autoboot in 0 seconds
[   1.191] switch to partitions #0, OK
[   1.191] mmc2(part 0) is current device
[   1.426] 61926804 bytes read in 213 ms (277.3 MiB/s)
[   1.428] ## Loading kernel from FIT Image at 08000000 ...
[   1.434]    Using 'conf' configuration
[   1.437]    Verifying Hash Integrity ... OK
[   1.441]    Trying 'kernel' kernel subimage
[   1.445]      Description:  Linux kernel
[   1.449]      Type:         Kernel Image
[   1.453]      Compression:  uncompressed
[   1.457]      Data Start:   0x08000388
[   1.460]      Data Size:    34631680 Bytes = 33 MiB
[   1.465]      Architecture: RISC-V
[   1.468]      OS:           Linux
[   1.472]      Load Address: 0x00600000
[   1.475]      Entry Point:  0x00600000
[   1.479]      Hash algo:    sha256
[   1.482]      Hash value:
8209e4097bd7b277583c306b9a9cd3e60cb74653b97a07b29b59d26187c45772
[   1.491]    Verifying Hash Integrity ... sha256+ OK
[   2.083] ## Loading ramdisk from FIT Image at 08000000 ...
[   2.086]    Using 'conf' configuration
[   2.089]    Verifying Hash Integrity ... OK
[   2.094]    Trying 'ramdisk' ramdisk subimage
[   2.098]      Description:  Initramfs
[   2.101]      Type:         RAMDisk Image
[   2.105]      Compression:  zstd compressed
[   2.109]      Data Start:   0x0a11ce98
[   2.113]      Data Size:    27203605 Bytes = 25.9 MiB
[   2.118]      Architecture: RISC-V
[   2.121]      OS:           Linux
[   2.124]      Load Address: unavailable
[   2.128]      Entry Point:  unavailable
[   2.132]      Hash algo:    sha256
[   2.135]      Hash value:
21357d3c616a994d04888d1d10aa110bd13787c355f75e3f0ca1074f11accd3e
[   2.144]    Verifying Hash Integrity ... sha256+ OK
[   2.610] WARNING: 'compression' nodes for ramdisks are deprecated, please fix
your .its file!
[   2.616] ## Loading fdt from FIT Image at 08000000 ...
[   2.622]    Using 'conf' configuration
[   2.625]    Verifying Hash Integrity ... OK
[   2.629]    Trying 'fdt' fdt subimage
[   2.633]      Description:  Flattened Device Tree
[   2.637]      Type:         Flat Device Tree
[   2.641]      Compression:  uncompressed
[   2.645]      Data Start:   0x0a107484
[   2.649]      Data Size:    88384 Bytes = 86.3 KiB
```

```
[   2.654]      Architecture: RISC-V
[   2.657]      Hash algo:    sha256
[   2.660]      Hash value:
80b227399c508dcfaeb2e8e8d688a0b156b81e67b7c74d7b77f051c97a52584b
[   2.669]    Verifying Hash Integrity ... sha256+ OK
[   2.674]    Booting using the fdt blob at 0xa107484
[   2.678]    Loading Kernel Image
[   2.698]    Loading Ramdisk to 7c38a000, end 7dd7b815 ... OK
[   2.719]    Loading Device Tree to 000000007c371000, end 000000007c38993f ...
OK

Starting kernel ...

[    0.000000] Linux version 6.6.63 (root@bianbu-25.04-build-bsp-c48h8-pxdsc)
(gcc (Ubuntu 14.2.0-19ubuntu2bb1) 14.2.0, GNU ld (GNU Binutils for Ubuntu) 2.44)
#2.2.7.3 SMP PREEMPT Fri Aug 15 06:14:36 UTC 2025
...
```

Getting the Signature Checking to Actually Mean Something

In its current state, U-Boot will indiscriminately boot the .itb file. It will perform the hash checks, but won't do anything with the information. U-Boot and OpenSBI will need to be rebuilt. Unfortunately, there's no comprehensive information online on how they were built for the Bananapi F3, so work is being done to reverse engineer the configuration, add in a strict signature check so it won't boot unsigned kernels, and reflash the updated uboot.itb to the eMMC partition. The following GitHub repositories may be of use here:

```
git clone https://github.com/u-boot/u-boot.git
git clone https://github.com/cyyself/opensbi -b k1-opensbi
```

In OpenSBI, run `make PLATFORM=generic`. In U-Boot, you'll want to modify `configs/bananapi-f3_defconfig` to contain whatever `CONFIG` statements allow you to enforce strict signature verification checks, and whatever else that works in pursuit of that goal. For the purpose of kernel signature checking, these settings are probably pertintent:

```
CONFIG_FIT=y
CONFIG_FIT_SIGNATURE=y
CONFIG_FIT_SIGNATURE_STRICT=y
CONFIG_RSA=y
CONFIG_RSA_VERIFY=y
```

```
CONFIG_SPL_RSA=y
CONFIG_SPL_SHA256=y
CONFIG_SPL_FIT_SIGNATURE=y
```

Once you've modified the file to your liking, you can rebuild using the following commands:

```
make distclean
make bananapi-f3_defconfig
make OPENSBI=../opensbi/build/platform/generic/firmware/fw_dynamic.bin
```

If you encounter any issues, you may want to consider pulling `fw_dynamic` from Bianbu. Further research needs to be done to see which file achieves the desired outcome. Hopefully, you don't have to modify any of U-Boot's board-specific `.dts` files...

The U-Boot partition lies on `/dev/mmcblk2p4`. Flash to it when ready:

```
dd if=u-boot.itb of=/dev/mmcblk2p4 bs=512
```

If this doesn't boot the signed kernel correctly, you'll either get a horrible error telling you to reset the board (over UART) or you'll be dropped to a U-Boot shell. Play around in the shell if you'd like, or insert the SD card and press the onboard reset button to go back to Bianbu. Once you succesfully boot Bianbu, reflash the U-boot from the SD card as follows:

```
dd if=/dev/mmcblk0p4 of=/dev/mmcblk2p4 bs=512
```

And reboot the system without the SD card. You should be back to Debian. Repeat this process ad infinitum, and you have a summary of my Thanksgiving break.